

Managing systemd Services and Targets



Andrew Mallett

Author and Trainer

@theurbanpenguin www.theurbanpenguin.com



Overview



Systemd Based Linux Systems:

- uniformity
- eco-system
- services
- targets
- logs and journalctl



Systemd Eco-system

```
# hostnamectl set-hostname  
# timedatectl set-timezone 'Europe/London'  
# localectl set-locale LANG=en_GB.utf8
```

Demo



Linux Uniformity:

- hostnamectl
- timedatectl
- localectl



PID 1 = Systemd

Boot loader

Kernel

Systemd



systemd-analyze

Analyze Boot Time

```
# systemd-analyze
```

```
# systemd-analyzed blame
```

Demo



Analyze System Startup:

- process ID 1
- systemd-analyze



Using Systemctl to Manage Services

systemctl

```
# systemctl status cron
```

```
# systemctl enable --now cron
```

```
# systemctl disable --now cron
```

```
# systemctl cat cron
```

```
# systemctl edit cron --full
```


Demo



Managing Services:

- systemctl
- start/stop
- enable/disable
- --now
- mask
- status
- cat / edit

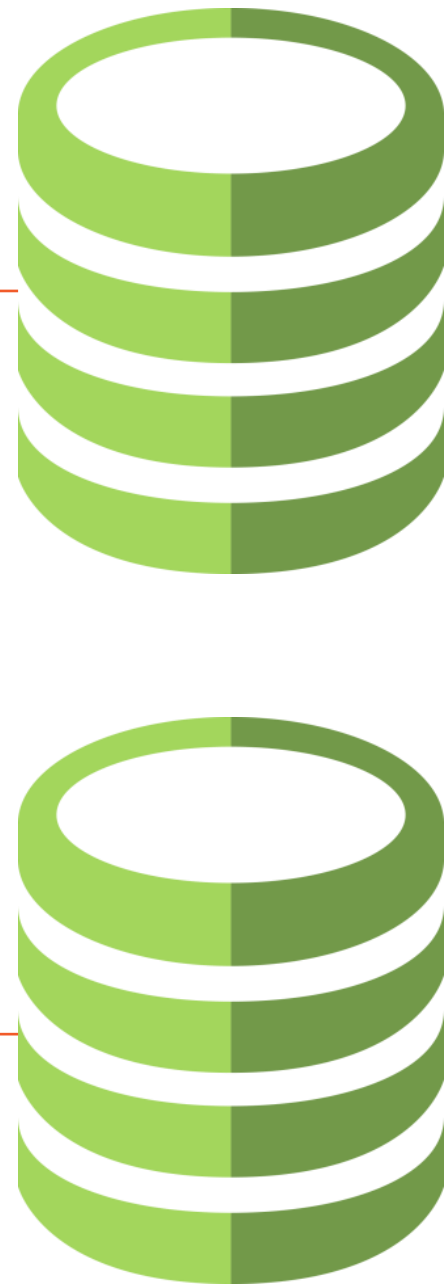


```
$ sudo -i
# fallocate -l 1G /root/disk1
# losetup /dev/loop10 /root/disk1
# parted /dev/loop10 mklabel msdos mkpart primary 0% 100%
# lsblk
```

Scenario

Creating our own disk file for use as storage in Ubuntu is easy. We use fallocate to create the file. Although the file is persistent it will need to be linked using losetup each time on startup and the partition table read with partprobe. Creating a service unit can automate the tasks for us.

The Problem with.....



The loop devices we have used do not persist

Creating a `service unit` we can script the device creation system start-up

Running both `losetup` and `partprobe`



losetup.service

[Unit]

Description=Setup Loop Device

DefaultDependencies=no

Before=local-fs.target

After=systemd-udev.service

[Service]

Type=oneshot

ExecStart=/sbin/losetup /dev/loop1 /root/disk1

ExecStart=/sbin/partprobe /dev/loop1

RemainAfterExit=no

[Install]

WantedBy=local-fs.target

Demo



Working with Disk Files and Service Units:

- fallocate
- losetup
- parted
- systemctl edit




```
$ systemctl list-units --type target
$ runlevel
$ systemctl get-default
$ sudo systemctl isolate multi-user
$ runlevel
$ sudo systemctl set-default multi-user
$ systemctl get-default
```

Targets

Targets group services together, a service is configured to be "installed" into a target if it is enabled for auto-start. Targets can include other targets allowing the grouping to become more global. For example, the graphical target will include the multi-user target and hence, all multi-user services will also start in the graphical target

Demo



Working with Targets



```
$ ls -l /var/log  
$ cat /etc/rsyslog.conf
```

Traditional Logs

Traditionally, logs are stored in individual log files below `/var/log`. The main log file in Ubuntu is `/var/log/syslog`. The `/etc/rsyslog.conf` controls where log messages are sent to, along with the `/etc/rsyslog.conf.d/` subdirectory.




```
$ sudo journalctl  
$ sudo journalctl --unit ssh  
$ sudo journalctl --unit ssh --since today
```

Journal Log

The systemd journal log amalgamates logs centrally and can be filtered using journalctl



```
$ sudo grep -i 'storage' /etc/systemd/journald.conf
$ man 5 journald.conf
$ sudo sed -i 's/#Storage=auto/Storage=persistent/' /etc/systemd/journald.conf
$ sudo grep -i 'storage' /etc/systemd/journald.conf
$ sudo systemctl restart systemd-journald
```

Persisting Journal Logs

The journal log may only be resident in memory, we can configure persistent storage if required.



Demo



Working with Logs:

- viewing logs and the tail command
- viewing the journal log
- persisting the journal log



Summary



Managing Services

- systemd and uniformity
 - hostnamectl
 - localectl
 - timedatectl
- managing services using systemctl
- creating service units
- managing targets
- managing logs and journalctl



Up Next:
Scripting Automation in Linux

